

UNIVERSIDAD RAFAEL LANDIVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

MANEJO E IMPLEMENTACION DE ARCHIVOS



“Proyecto 1: Sistema de Control de Archivos”

ANGELO GUSTAVO GARCÍA IXCAMPARY – 1547225

QUETZALTENANGO, 16 DE SEPTIEMBRE DEL 2026

1. Selección y Justificación Técnica del Formato de Almacenamiento

Para la persistencia de la configuración de usuario se seleccionó el formato **JSON** (`config.json`) frente a la alternativa descartada **CSV** (`config.csv`). A continuación se detallan los criterios técnicos que justifican esta elección:

Tabla Comparativa: JSON vs. CSV

Criterio Técnico	Formato JSON (Elegido)	Formato CSV (Descartado)
Preservación de Tipos de Datos	Soporta nativamente tipos numéricos (<code>tamano_fuente: 12</code>), cadenas y booleanos. No requiere conversiones manuales al leer.	Trata todos los campos como cadenas planas (<code>string</code>). Requeriría parsear y castear manualmente tipos enteros o arreglos.
Facilidad de Parseo y Mantenimiento	Mapeo directo <code>\$1:1\$</code> a diccionarios en Python mediante el módulo nativo <code>json</code> . Carga y guardado en 1 línea.	Diseñado para datos tabulares (filas/columnas). Estructurar datos de configuración jerárquicos en CSV resulta forzado e ineficiente.
Manejo de Rutas y Caracteres Especiales	Escapa caracteres de manera transparente. Las rutas con comas, diagonales o comillas no corrompen el esquema.	Sensible a caracteres delimitadores (comas, punto y coma). Una ruta de archivo con comas podría romper la columna esperada.
Robustez ante Corrupción	La estructura de llaves <code>{ }</code> permite validación rápida de sintaxis. Si falta una coma o llave, se detecta antes de alterar variables.	Si falta un valor en una fila, los datos se desplazan de columna silenciosamente, asignando un color a la variable de fuente.
Legibilidad Humana	Altamente legible y auto-descriptivo gracias al formato clave-valor con sangría.	Dificultad de lectura cuando las llaves están en la cabecera y los valores en la segunda fila sin contexto visual claro.

Justificación Final:

Se seleccionó **JSON** por ser el estándar de la industria para la gestión de configuraciones ligeras. Su compatibilidad con la estructura de diccionarios en Python simplifica la manipulación en memoria, soporta heterogeneidad de tipos de datos y facilita la validación de claves requeridas antes de actualizar los componentes de la interfaz gráfica.

2. Implementación de Escritura Segura y Respaldo (Backup)

Para garantizar la integridad total de los datos y prevenir archivos corruptos o incompletos ante fallos imprevistos del sistema o interrupciones de energía, se implementó un flujo de **Persistencia Segura en Tres Fases** dentro del módulo `core/config_manager.py`:

1. **Creación Preventiva del Respaldo (.bak):** Antes de modificar el archivo principal activo (`config.json`), se realiza una copia idéntica hacia `config.bak` utilizando la función `shutil.copy2()`. Esto resguarda el último estado válido conocido.
2. **Escritura Temporal Segura con Vaciado a Disco (.tmp):** Se escribe la nueva configuración en un archivo borrador `config.tmp`. Para asegurar que los datos no queden atrapados en el buffer de memoria del sistema operativo, se fuerza la escritura física en disco invocando `f.flush()` seguido de `os.fsync(f.fileno())`.
3. **Reemplazo Atómico (`os.replace`):** Finalmente, se reemplaza `config.tmp` por `config.json`. La función `os.replace` realiza una operación atómica a nivel del sistema operativo, garantizando que el archivo destino cambie en un solo paso y evitando que el archivo activo quede corrupto.

Fragmento de Código Relevante (`core/config_manager.py`)

```
# 1. Respaldo (.bak) de la configuración previa, si existe un archivo activo
if os.path.exists(CONFIG_FILE):
    try:
        shutil.copy2(CONFIG_FILE, BACKUP_FILE)
        logging.info(f"Respaldo creado con éxito en '{BACKUP_FILE}'.")
    except Exception as e:
        backup_ok = False
        logging.warning(f"No se pudo crear el archivo de respaldo: {e}")

    try:
        # 2. Escritura segura en archivo temporal (.tmp) en UTF-8
        with open(TEMP_FILE, "w", encoding="utf-8") as f:
            json.dump(nueva_config, f, ensure_ascii=False, indent=4)
            f.flush()
            os.fsync(f.fileno()) # fuerza la escritura a disco antes de renombrar

        # 3. Reemplazo atómico del archivo final
        os.replace(TEMP_FILE, CONFIG_FILE)
        logging.info("Configuración guardada y reemplazada atómicamente.")
```

3. Manejo de Errores Exigidos y Codificación UTF-8

La aplicación cuenta con mecanismos de defensa para evitar caídas del programa (*tracebacks* no capturados), degradando el comportamiento de forma elegante:

- **Archivo Ausente:** Se verifica la existencia del archivo mediante `os.path.exists(CONFIG_FILE)`. Si el archivo no existe al iniciar, la aplicación carga una copia en memoria de `DEFAULT_CONFIG` e informa al usuario. El archivo físico se genera automáticamente en el primer guardado.
- **Archivo Corrupto o Formato Inválido:** Se capturan las excepciones `json.JSONDecodeError` y `ValueError`. Además, el método `validar_estructura()` comprueba la existencia y el tipo de dato de cada clave esperada. En caso de inconsistencia, se notifican los valores por defecto sin interrumpir la ejecución.
- **Falta de Permisos (Lectura/Escritura):** Se captura explícitamente mediante `PermissionError`. Si no hay permisos de lectura, se recurre a la configuración por defecto; si no hay permisos de escritura, la operación se aborta, se elimina el archivo borrador temporal `.tmp` y se notifica al usuario mediante un diálogo de alerta (`QMessageBox.critical`).
- **Codificación UTF-8:** Todas las aperturas de archivo establecen explícitamente `encoding="utf-8"`. El uso del parámetro `ensure_ascii=False` en `json.dump` permite almacenar tildes y la letra ñ directamente en caracteres legibles sin secuencias de escape unicode.

4. Ejemplos de Archivos Generados

Ejemplo de Archivo Activo (`config.json`)

```
{
  "nombre_usuario": "Programador Snior.Melqui",
  "tema_interfaz": "claro",
  "idioma": "es-ES",
  "tamano_fuente": 20,
  "color_menu": "#aaffff",
  "color_letra": "#000000",
  "foto_perfil": "C:/Users/27gar/Pictures/WhatsApp Image 2026-03-24 at 08.31.15.jpeg"
}
```

Ejemplo de Archivo de Respaldo (`config.bak`)

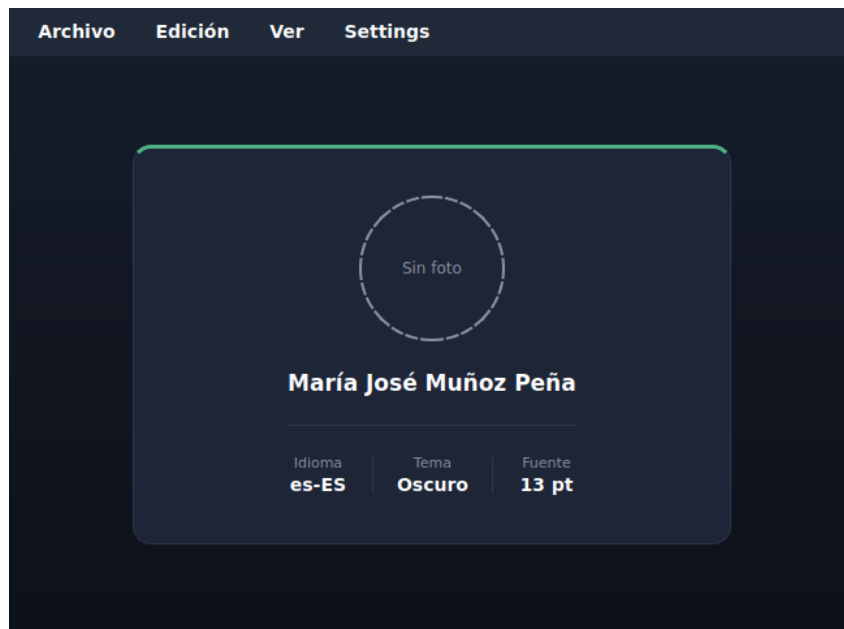
```

{
  "nombre_usuario": "Programador Snior.Melqui",
  "tema_interfaz": "claro",
  "idioma": "es-ES",
  "tamano_fuente": 26,
  "color_menu": "#aaffff",
  "color_letra": "#000000",
  "foto_perfil": "C:/Users/27gar/Pictures/WhatsApp Image 2026-03-24 at 08.31.15.jpeg"
}

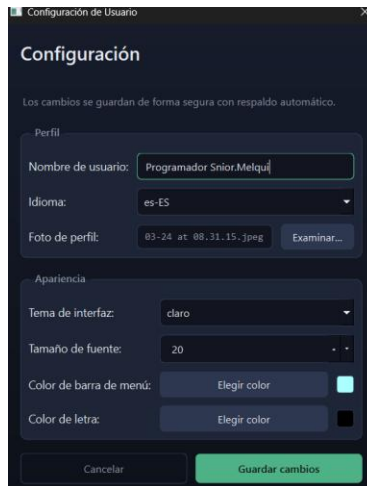
```

5. Evidencias de Funcionamiento (Capturas de Pantalla)

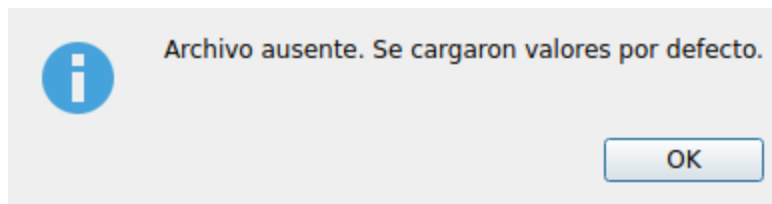
Descripción: Muestra la interfaz principal ejecutándose con la tarjeta de perfil, la foto recortada en formato circular y los estilos aplicados.



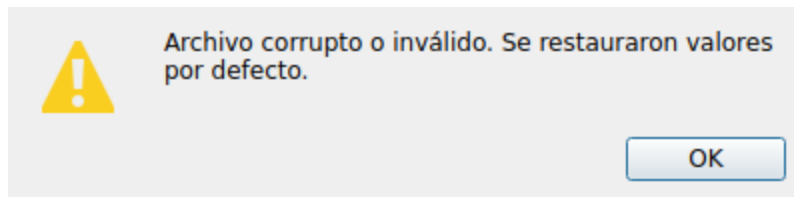
Descripción: Muestra el diálogo emergente de Settings con los campos de entrada, selectores de color nativos y el explorador de archivos.



Descripción: Captura del mensaje informativo emitido por la aplicación al iniciar sin un archivo config.json presente en el sistema.



Descripción: Captura del aviso de alerta al detectar un archivo JSON mal formado o con datos inválidos, restaurando la configuración por defecto.



Descripción: Muestra la carpeta del proyecto en el sistema operativo evidenciando la coexistencia de config.json y config.bak.

